

Лабораторная работа №2. Разработка тест-кейсов

ЗАДАНИЕ

1. Составить набор тест-кейсов для тестирования простого одноразового карандаша с деревянной оправой грифеля. Достаточно привести только уникальный идентификатор и цель (идею, краткое описание) для каждого тест-кейса. Количество тест-кейсов: 20 – 25.
2. Составить набор тест-кейсов для тестирования следующего сценария требования к web приложению по продаже книг: **система проводит аутентификацию пользователя.**
3. Составить набор тест-кейсов для тестирования следующего требования к web приложению по продаже книг: **система проводит авторизацию пользователя с разделением и автоматическим назначением привилегий для различных категорий: «пользователь» (покупатель/продавец), «администратор».**
4. Составить набор тест-кейсов для тестирования следующего сценария использования (use case) web приложения по продаже книг: администратор добавляет в систему новый тип товара *Электронные книги*, отличающиеся наличием атрибутов, например, таких как формат, размер.
5. Составить набор тест-кейсов для тестирования компонента Microsoft Windows «Калькулятор» (calc.exe) для режима работы «Обычный». Достаточно привести только уникальный идентификатор и цель (идею, краткое описание) для каждого тест-кейса. Количество тест-кейсов: 20 – 25.
6. Составить набор тест-кейсов для тестирования сервиса Яндекс.Погода (<http://pogoda.yandex.ru>). Достаточно привести только уникальный идентификатор и цель (идею, краткое описание) для каждого тест-кейса. Количество тест-кейсов: 20 – 25.
7. Составить набор тест-кейсов для тестирования команды командной строки cd, предназначенной для изменения текущего рабочего каталога в Unix, DOS и других операционных системах. Достаточно привести только

уникальный идентификатор и цель (идею, краткое описание) для каждого тест-кейса. Количество тест-кейсов: 20 – 25.

СВЕДЕНИЯ ПО СОСТАВЛЕНИЮ ТЕСТ-КЕЙСОВ (С ПРИМЕРАМИ)

Источник - Савин Р. Тестирование Дот Ком, или Пособие по жестокому обращению с багами в интернет-стартапах. — М.: Дело, 2007. — 312 с.

Структура тест-кейса

Для нахождения бага (что является смыслом любого тестирования) кроме ожидаемого нужен и фактический результат. Как правило, необходима инструкция, как **прийти** к фактическому результату.

Пример

Допустим, тестировщику А. Боброву, который только что начал работать в нашем стартапе www.testshop.rs, дали для исполнения следующий тест-кейс:

"Оплата может быть произведена картой VISA". Сразу же возникает по крайней мере две проблемы:

- для исполнения тест-кейса нужна тестировочная карта VISA,
- которой у него нет;
- он не знает, как проверить, был ли действительно осуществлен платеж, даже если бы у него была карта.

Единственное, что более или менее понятно, — это процесс покупки в интернет-магазине (найти товар, добавить в корзину и т.д.), что в данной ситуации помогает немного.

Пример

Допустим, тестировщику А. Боброву, который только что начал работать в нашем стартапе www.testshop.rs, дали для исполнения следующий тест-кейс: Шаги:

1. *Открой www.main.testshop.rs*

2. *Введи в поле "Имя пользователя": "testuser1"*
3. *Введи в поле "Пароль": "pa\$\$wOrd"*
4. *Нажми кнопку "Войти"*
5. *Введи в поле "Поиск": "book117"*
6. *Нажми кнопку "Найти"*
7. *Кликни линк "Добавить в корзину"*
8. *Кликни линк "Корзина"*
9. *Кликни линк "Оплатить"*
10. *Выбери из меню "Вид карты": "VISA"*
11. *Введи в поле "Номер карты": "9999-5148-2222-1277"*
12. *Введи в поле "Действительна до": "12/07"*
13. *Введи в поле "CW2": "778"*
14. *Нажми кнопку "Завершить заказ"*
15. *Запиши номер заказа ____*
16. *Запроси базу данных:*

select result from cc_transaction where id = <номер заказа >; Ожидаемый результат: "10"

В последнем примере (который мы назовем тест-кейс с картой) к **ожидаемому результату (ОР)** добавились **шаги (steps)**, которые должны привести нас к **фактическому результату (ФР)**, необходимому, чтобы узнать, есть баг или нет. Совокупность шагов называется **процедурой (procedure)**.

Если провести аналогию, то

- **шаги** — это ступеньки лестницы;
- **ожидаемый результат** — это некий предмет, который мы **должны найти**, если поднимемся по этим ступенькам;
- **фактический результат** — это то, **что мы реально нашли** после того, как поднялись по этим ступенькам.

ИСХОДЯ ИЗ ОСНОВНОЙ компьютерной концепции ВВОД/ВЫВОД (на языке оригинала — input/output):

- *шаги* — это инструкция по вводу;
- *исполнение шагов* — это ввод;
- *ожидаемый результат* — это ожидаемый вывод;
- *фактический результат* — это фактический вывод.

Исполнение тест-кейса завершается сравнением вывода фактического и вывода ожидаемого.

Исход исполнения тест-кейса (*testcase result*)

Каждый тест-кейс, исполнение которого завершено, дает нам одно из двух:

- 1. Положительный исход (*PASS*),** если ФР равен ОР, либо
- 2. Отрицательный исход (*FAIL*),** если ФР не равен ОР: **найден баг!**

Иногда возникает ситуация, когда мы заблокированы (*test case execution is blocked*), так как не можем пройти ВСЕ шаги тест-кейса. Например, мы не можем продвинуться дальше, если кнопки "Завершить заказ" из шага 14 не существует на соответствующей веб-странице. В таком случае мы рапортуем баг (в данном случае баг об отсутствии кнопки "Завершить заказ") и откладываем исполнение тест-кейса до устранения бага.

Полезные атрибуты тест-кейса

УНИКАЛЬНЫЙ ID (*Unique ID*)

ID должен быть уникальным в пределах не только документа, содержащего тест-кейс (об этом документе позже), но и всего департамента качества. Рациональное обоснование: со временем появится необходимость вести статистику по тест-кейсам, обновлять, удалять или переносить в другой документ некоторые из них, прикрывать спину и т.д.

ПРИОРИТЕТ ТЕСТ-КЕЙСА (*Test Case Priority*)

Это важность тест-кейса. Важность отражается по шкале от 1 до *n*, где 1 — это высший приоритет, а *n* — это низший приоритет. Допустим, тест-кейс,

проверяющий, работает ли кнопка "Купить", будет 1-го приоритета, а тест-кейс, проверяющий цвет шрифта линка "Гостевая книга", будет 4-го приоритета.

Допустим, у нас есть два тест-кейса: один 1-го приоритета и другой — 3-го приоритета, оба тестируют некую функциональность *A*, и есть время для исполнения только одного из них. Естественно, что мы выберем тест-кейс 1-го приоритета. Приоритезация тест-кейсов особо полезна при регрессивном тестировании.

Как правило, автор тест-кейса решает, насколько жизненно важна, определяющая и критична вещь, проверяемая данным тест-кейсом.

ИДЕЯ (IDEA)

Это **описание** конкретной вещи, проверяемой тест-кейсом (в дальнейшем эту конкретную вещь мы также будем называть "идея тест-кейса").

Пример

В тест-кейсе с картой ожидаемым результатом является значение "10" в колонке result строки с нашей транзакцией. Поймет ли, ЧТО мы тестируем, человек, который не знает, что программисты www.testshop.rs обозначают первую цифру результата транзакции индексом кредитной карты (где "1" — это VISA, "2" — MasterCard, "3" — Switch), а вторую — флагом успеха (где "0" — это успех, а "1" — ошибка) и соответственно "10" означает, что транзакция с картой VISA была успешной?

Дело в том, что "непосвященным" может стать даже автор тест-кейса, скажем, через месяц после написания. Поэтому **в начале** тест-кейса следует написать доступным и понятным широкому кругу людей языком: "Оплата может быть произведена картой VISA", чтобы любой, кто возьмет этот тест-кейс в руки, не ломал голову, а сразу понял, что проверяется этим тест-кейсом.

ПОДГОТОВИТЕЛЬНАЯ ЧАСТЬ (SETUP and ADDITIONAL INFO)

В подготовительную часть тест-кейса могут включаться:

- данные о существующем экаунте пользователя (*legacy user account*) или инструкции по созданию нового экаунта (*newuser account*);
- другие данные, используемые в тест-кейсе, например атрибуты используемой кредитной карты;
- запросы к базе данных (*SQL queries*), используемые в тест-кейсе;
- комментарии в помощь тестировщику, например о нюансах, которые могут встретиться при исполнении тест- кейса;
- другие вещи, облегчающие **исполнение** и **поддержку** тест-кейса

ИСТОРИЯ РЕДАКТИРОВАНИЯ (*RevisionHistory*)

Для того чтобы иметь сведения о создании и истории развития каждого тест-кейса, ведется краткий журнал изменений, где отражаем: **Кто? Что? Зачем? Когда? Почему?**

Атрибуты истории редактирования:

- **Created on** <date> by <name> — Тест-кейс создан <дата><кем>;
- **Modified on** <date> by <name> — Тест-кейс изменен <дата><кем>;
- **Change** — Что, зачем и почему было изменено.

Для этого заполняется значение атрибута в поле справа от "*Created on...*" или "*Modified on...*".

Давайте создадим тест-кейс с картой, используя только что полученные знания по полезным атрибутам тест-кейса:

TC ID/Priority	CCPG0001	1
IDEA: Оплата может быть произведена картой VISA SETUP and ADDITIONAL INFO: Аккаунт: testuser1/paSSwOrd Наименование товара: book117 Данные карты: Номер: 9999-5148-2222-1277		

Окончание действия: 12/07 CVV2: 778	
SQL1: select result from cc_transaction where id = <номер заказа>;	
Revision History	
Created on: 11/17/2003 by	Новый тест-кейс
О.Тарасов	

Execution part	
PROCEDURE	EXPECTED RESULT
1. Открой www.main.testshop.rs 2. Введи имя пользователя. 3. Введи пароль. 4. Нажми кнопку "Войти". 5. Введи наименование товара в поле поиска. 6. Нажми кнопку "Найти". 7. Кликни линк "Добавить в корзину". 8. Кликни линк "Корзина". 9. Кликни линк "Оплатить". 10. Выбери вид карты. 11. Введи номер карты. 12. Введи срок окончания действия. 13. Введи CVV2. 14. Нажми кнопку "Завершить заказ". 15. Запиши номер заказа	> "10"
16. Запроси базу данных с SQL1 и запиши результат	

Тест-кейсы, управляемые данными

Основной плюс нового тест-кейса с картой заключается в том, что нам не нужно вносить изменения в ШАГИ, чтобы протестировать по тому же сценарию другие карты. Единственное, что нам нужно, — это модифицировать исходные ДАННЫЕ.

Таким образом, если кроме VISA нам нужно протестировать по тому же сценарию еще две карты, то мы в каждом из новых тест-кейсов переписываем только пять подчеркнутых значений, проживающих в шапке тест-кейса и секции ожидаемого результата (меняем и *ID* тест-кейса — *TC ID*, который, как мы помним, должен быть всегда уникальным):

VISA

9999-5148-2222-1277

12/07

778

10

Такой вид тест-кейса называется ***data-driven*** ("управляемый данными"), т.е. **когда данные и инструкции по их применению не смешаны, а разделены и слинкованы.**

Поддерживаемость тест-кейса

Часто возникает проблема с тем, что, например, веб-сайт, а конкретно интерфейс пользователя (*User Interface*), очень часто меняется.

Пример

Кнопка "Войти" из шага 4 легко может быть переименована во "Вход". Следовательно, если у нас есть 3 тест-кейса, то нужно внести 3 изменения. А что, если у нас 500 тест-кейсов, где упоминается кнопка "Войти", и эти тест-кейсы разбросаны по разным документам и таких маленьких изменений будут

десятки, и постепенно ваши тест-кейсы буду потреблять на поддержку много времени.

Пример

А что, если не имя кнопки, а сам путь, по которому вы добираетесь до фактического результата, претерпел изменения? Например, шаги 7 и 9 станет разделять не линк "Корзина", а еще несколько дополнительных линков и кнопок, появившихся в новой версии www.testshop.rs.

Данная проблема носит название — ***maintainability*** (поддерживаемость), т.е. **насколько легко и просто можно изменить тест-кейс при изменениях в ПО.**

Если мы разобьем шаги нашего нового тест-кейса с картой на логические модули, получим:

1. *Вход в систему* (логин — *log in*).
2. *Поиск товара.*
3. *Добавление товара в корзину.*
4. *Оплата.*
5. *Фиксация номера заказа.*
6. *Запрос базы данных.*

Почему бы нам не выбросить из тест-кейса детали по следующим позициям?

1. Вход в систему

В общем-то можно догадаться, куда ввести имя пользователя, куда пароль и на какую кнопку нажать, тем более что в данном случае мы не тестируем процесс логина, это было или будет сделано при исполнении соответствующего тест-кейса, сейчас мы просто используем логин.

2. Поиск товара

Все из предыдущего пункта применимо и здесь. Кроме того, допустим, что *book117* была удалена из нашей базы данных. В таком случае нужно

превентировать такую ситуацию тем, что не будем давать имени конкретного товара. Что найдется, то найдется (так как то, что найдется, **в данном случае значения не имеет**).

3. Добавление товара в корзину

Концепция из "1. Вход в систему" применима и здесь.

4. Оплата

Здесь появляется другая проблема: если мы производим оплату в сотнях тест-кейсов, т.е. сотни раз включаем в тест-кейс те же семь шагов (8—14 включительно), то при изменении даже в одном из этих шагов нам придется переписывать эти сотни тест-кейсов.

Не проще ли вынести шаги, повторяющиеся от тест-кейса к тест-кейсу, во внешний документ и вместо них включить в тест-кейс лишь один шаг-ссылку «Произведи ОПЛАТУ КАРТОЙ из секции "SETUP and ADDITIONAL INFO"»? Поступив таким образом, мы сэкономим громадное количество часов рабочего времени, так как при необходимости менять шаги нужно будет только в одном месте.

***Кстати,** "оплата картой" — это линк к страничке в локальной сети с соответствующей инструкцией, называемой, например, "Как произвести оплату кредитной картой".*

***Кстати,** хорошей идеей является создание в локальной сети вашей компании мини-веб-сайта департамента качества, где наряду с веб-страничками с*

- контактной информацией работников департамента,*
- тинками к файлам с тест-комплектами,*
- другой полезной информацией*

*расположится и внутреннее **Пособие для тестировщиков** (QA Knowledge Base), где кроме прочего будут задокументированы повторяющиеся сценарии.*

Теперь обобщим уже известные нам мероприятия по улучшению поддерживаемости тест-кейса:

1. Сделать тест-кейс *data-driven*.
2. Не описывать шаги по явно очевидным сценариям (например, логин).
3. Не давать конкретных деталей, если они не играют роли при исполнении тест-кейса (например, имя товара).
4. Вынести во внешний документ повторяющиеся сценарии (например, семь шагов оплаты).

TC ID/Priority	CCPG0001	1
IDEA: Оплата может быть произведена картой VISA SETUP and ADDITIONAL INFO: Аккаунт: testuser1/paSSwOrd Данные карты: Номер: 9999-5148-2222-1277 Окончание действия: 12/07 CVV2: 778 SQL1: select result from cc transaction where id = <номер заказа>;		
Revision History		
Created on: 11/17/2003 by		Новый тест-кейс
О.Тарасов		
Modified on: 11/26/2003 by И.		Шаги были упрощены, чтобы сделать тест-
Новикова		кейс более удобным для поддержки

Execution part	
PROCEDURE	EXPECTED RESULT
1.Открой www.main.testshop.rs 2. Войди в систему. 3. Найди любой товар. 4.Добавь товар в корзину.	> "10"

5. Произведи оплату картой из секции SETUP and ADDITIONAL INFO	
6. Запиши номер заказа	
7. Запроси базу данных с SQL1 и запиши результат	

Тест-кейсом проверятся только одна конкретная вещь, и в идеальном варианте для проверки этой вещи достаточно предусмотреть в тест-кейсе только один *ОР*.

*Допустим, что в соответствии с пунктом 12.6 документа "Дизайн кода для спецификации #6522" признаком того, что оплата была успешно проведена картой VISA, будет **одновременное** наличие не одного, а двух условий:*

- 1 Значение "10" в соответствующей колонке соответствующей строки в базе данных.*
- 2 Уменьшение баланса на счете с картой VISA на сумму, равную сумме оплаты.*

То есть получается, что для тестирования одной вещи ("Оплата может быть произведена картой VISA") нужно проверить соответствие жизненной реальности двум ожидаемым результатам.

У нас есть два пути:

1. Разложить идею тест-кейса на две идеи и создать два тест-кейса.
2. Оставить идею тест-кейса неприкосновенной и включить в один тест-кейс два *ОР*, т.е. у нас складывается ситуация, когда исполнение тест-кейса будет иметь положительный исход, только если ОБА фактических результата совпадут с соответствующими им ожидаемыми результатами.

Вот как будет выглядеть визуально путь 2:

TC ID/Priority	CCPG0001	1
IDEA: Оплата может быть произведена картой VISA SETUP and ADDITIONAL INFO: Аккаунт: testuser1/paSSwOrd Данные карты: Номер: 9999-5148-2222-1277 Окончание действия: 12/07 CVV2: 778 SQL1: select result from cc transaction where id = <номер заказа>; Баланс счета карты можно посмотреть здесь: www.main.testshop.rs/1277/balance.htm		
Revision History		
Created on: 11/17/2003 by		Новый тест-кейс
О.Тарасов		
Modified on: 11/26/2003 by		Шаги были упрощены, чтобы сделать тест-кейс
И. Новикова		более удобным для поддержки
Modified on: 01/17/2003 by		Изменение шагов и второй ожидаемый
И. Новикова		результат с целью удостоверения в снятии денег со счета
Execution part		
PROCEDURE		EXPECTED RESULT
1. Запиши баланс счета карты 2. Открой www.main.testshop.rs 3. Войди в систему. 4. Найди любой товар. 5. Добавь товар в корзину. 6. Произведи <i>оплату</i> <i>картой</i> из секции SETUP		S> "10"

and ADDITIONAL INFO (!!! запиши полную сумму заказа:). 7. Запиши номер заказа 8; Запроси базу данных с SQL1.	
9. Запиши баланс счета карты	> Шаг 1-Шаг 6

Как будет проходить исполнение этого тест-кейса?

Прошли восемь шагов. Остановились. Проверили. Затем прошли девятый шаг. Остановились. Проверили.

Исход исполнения этого тест-кейса будет считаться положительным только при одновременной истинности двух условий:

1. ФР после исполнения шага 8 = "10" и
2. ФР после исполнения шага 9 = Шаг 1 - Шаг 6 (т.е. значение из Шага 1 минус значение из Шага 6).

В теории лучше было бы разбить нашу идею тест-кейса на две части и создать два отдельных тест-кейса:

1. *IDEA*: "Правильное значение вставляется в базу данных при использовании VISA".
2. *IDEA*: "Верная сумма списывается с баланса карты".

И если есть возможность, то ЛУЧШЕ сделать именно два тест-кейса, НО на практике во многих случаях имеет смысл включить в тест-кейс 2 или больше ОР, так как:

- у вас может просто не быть времени на написание, исполнение и поддержку двух тест-кейсов*;
- сэкономленное время можно потратить на написание, исполнение и поддержку тест-кейса, которым мы бы проверили другую вещь**.

**Если у нас есть один случай, когда можно совместить два ОР, то написание, исполнение и поддержка двух тест-кейсов не представляет труда. А что, если у нас появляются сотни дополнительных тест-кейсов?..*

***В результате такой экономии мы с течением времени создаем десятки новых тест-кейсов, которые помогают нам провести более тщательное тестирование.*

Во многих случаях, когда несколько ожидаемых результатов просятся в один тест-кейс, нужно проверить

- значение(-я) на веб-странице и
- значение(-я) в базе данных,

т.е. нужна проверка **снаружи и изнутри** или *на front end и back end*.

В последнем примере мы непосредственно "разговаривали"

- с фронт-энд ом — в шаге 5, когда добавляли товар в корзину;
- с бэк-эндом — в шаге 8, когда запрашивали базу данных.

Проблемные тест-кейсы

Теперь посмотрим, какие недостатки вы должны убирать из своих тест-кейсов:

1. Зависимость тест-кейсов друг от друга.
2. Нечеткая формулировка шагов.
3. Нечеткая формулировка идеи и/или ожидаемого результата.

1. ЗАВИСИМОСТЬ ТЕСТ-КЕЙСОВ ДРУГ ОТ ДРУГА

Независимость тест-кейса выражается в том, что он не связан с другими тест-кейсами.

Другим распространенным случаем является допущение, что ПО или база данных уже приведены к нужному состоянию, так как были исполнены предыдущие тест-кейсы.

Пример

В тест-кейсе X мы создаем транзакцию покупки книги. В тест-кейсе Y мы, допуская, что тест-кейс X был успешно исполнен, проверяем атрибут успешности транзакции покупки книги, не создавая саму транз-акцию ("Зачем напрягаться, когда она уже создана?"). В итоге может произойти ситуация, когда транзакция покупки книги не создана, так как

- тест-кейс X был удален;*
- тест-кейс X был модифицирован так, что он создает транзакцию другого типа;*
- тест-кейс X не создал транзакции по объективной причине (например, не работал соответствующий код).*

Как результат, во всех трех случаях мы не можем исполнить тест-кейс Y, так как данных, на которые он опирается, просто не существует.

Таким образом, **хороший тест-кейс характеризуют:**

- отсутствие ссылок на другие тест-кейсы;**
- независимость от "следов", оставленных другими тест-кейсами в нашем ПО или базе данных.**

Следовательно, если у нас в документе A есть 10 тест-кейсов: тест-кейс 1, тест-кейс 2, ..., тест-кейс 10, то доказательством независимости каждого из тест-кейсов будет тот факт, что их без ущерба для тестирования можно **всегда исполнять в любом порядке**, например, тест-кейс 10, затем тест-кейс 2, затем тест-кейс 6 и т.д.

3. НЕЧЕТКАЯ ФОРМУЛИРОВКА ШАГОВ

Перечисляющиеся в тест-кейсе шаги должны быть объективно четкими и ясными.

Нужно помнить,

- то, что очевидно для вас сейчас, может стать совершенно непонятным через пару месяцев.**

Так, сокращенные шаги с нерасшифрованными аббревиатурами, понятными вам сейчас, могут впоследствии стать непонятными для вас самих, так что проще будет написать тест-кейс заново, чем разбираться в неосмотрительно сделанных описаниях;

- **тест-кейс, который не может быть исполнен никем, кроме его автора недопустим.**

Обоснование простое: что, если автор тест-кейса заболеет, уйдет в отпуск, уйдет из компании? Любой тест-кейс должен создаваться с мыслью о коллеге, который однажды возьмет его в руки.

Нужно избегать и другой крайности — когда шаги тест-кейса настолько детализируются, поскольку излишняя детализация ведет к усложнению поддерживаемости тест-кейса.

4. НЕЧЕТКАЯ ФОРМУЛИРОВКА ИДЕИ ТЕСТ-КЕЙСА ИЛИ ОЖИДАЕМОГО РЕЗУЛЬТАТА

Оба тезиса, о которых мы только что говорили:

- о том, что можно забыть то, что сейчас понятно, и
 - писать тест-кейсы нужно не для себя, а для всех —
- применимы и к идее и к ожидаемому результату. Нюансы для идеи тест-кейса и ожидаемого результата:

а. Не рекомендуется отсылка к внешнему документу.

Когда мы говорили о выноске части шагов в Пособие для тестировщиков, то делали это в случаях многократно повторяющихся сценариев, встречающихся в разных тест-комплектах, с целью сделать наш тест-кейс более поддерживаемым. С идеей же тест-кейса и ожидаемым результатом — совсем другая история.

Пример

Подумайте, удобно ли будет исполнять тест-кейс, если в секции IDEA напечатано:

«В этом тест-кейсе мы проверяем пункт 21.6 спецификации номер 34 "Сценарий добавления кредитной карточки к счету пользователя"»

или в секции Expected Result:

"Проверь, что значение последнего шага равно значению пересечения значения шага 5 по оси X и значению шага 23 по оси Y из таблицы 17.0 спецификации из секции IDEA"?

б. Нужно помнить, что суть секции *IDEA* — это ОБЪЯСНЕНИЕ идеи тест-кейса.

Пример

Если секция IDEA пуста или же в ней скромно напечатано "10", то каждый выполняющий этот тест-кейс каждый раз будет тратить несколько минут своего времени и/или времени своего коллеги на выяснение того, что же проверяется этим тест-кейсом.

в. Нужно помнить, что ожидаемый результат — это информация, на основании которой (вкупе с фактическим результатом) мы принимаем решение об исходе тест-кейса. Следовательно, точность и четкость в формулировке ожидаемого результата играют наиважнейшую роль.

Пример

Ожидаемый результат гласит: "Проверь, что показана страница с ошибкой", и страница с ошибкой действительно показывается. Дело в том, что если показывается не та ошибка, которая положена по спецификации, то будет пропущен баг. Почему он будет пропущен? Правильно: из-за неточной формулировки ожидаемого результата.

*Еще один **пример** плохого ожидаемого результата: "Все работает".*